

98. Validate Binary Search Tree Medium

Acceptance: 35.2% | [LeetCode](#)

Given the `root` of a binary tree, *determine if it is a valid binary search tree (BST)*.

A **valid BST** is defined as follows:

- The left subtree of a node contains only nodes with keys **strictly less than** the node's key.
- The right subtree of a node contains only nodes with keys **strictly greater than** the node's key.
- Both the left and right subtrees must also be binary search trees.

Example 1:

```
**Input:** root = [2,1,3]
**Output:** true
```

Example 2:

```
**Input:** root = [5,1,4,null,null,3,6]
**Output:** false
**Explanation:** The root node's value is 5 but its
right child's value is 4.
```

Constraints:

- The number of nodes in the tree is in the range `[1, 104]`.
- `-231 <= Node.val <= 231 - 1`

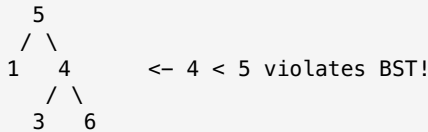
Claude Opus Solution

Validate Binary Search Tree

1. Intuition

The **key insight** is that a node being greater than its left child and less than its right child is **NOT enough**. We must validate against the *entire ancestry*, not just immediate parents.

Consider this tree:



Node `3` is less than its parent `4`, which looks fine locally. But `3` lies in the **right subtree of 5**, so it must be **greater than 5**. It isn't → invalid.

The pattern: Each node must fall within a valid **(min, max) range** that gets tightened as we descend: - Going **left** → the current node becomes the new **upper bound** (ceil). - Going **right** → the current node becomes the new **lower bound** (floor).

2. Approach (Pseudocode)

```
function isValidBST(node, floor=-inf, ceil=+inf):
    # Empty subtree is always valid
    if node is None:
        return True

    # Current node must lie strictly within (floor, ceil)
    if not (floor < node.val < ceil):
        return False

    # Recurse:
    # Left child: must be < node.val -> ceil becomes node.val
    # Right child: must be > node.val -> floor becomes node.val
    return isValidBST(node.left, floor, node.val)
    and isValidBST(node.right, node.val, ceil)
```

3. Python Solution

```
from typing import Optional
```

```

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def isValidBST(
        self,
        root: Optional[TreeNode],
        floor: float = float("-inf"), # exclusive lower bound for this subtree
        ceil: float = float("inf"), # exclusive upper bound for this subtree
    ) -> bool:
        # Base case: an empty node is trivially a valid BST
        if not root:
            return True

        # The current node's value MUST lie strictly inside (floor, ceil).
        # Strict inequalities enforce "strictly less/greater" BST rule.
        if not (floor < root.val < ceil):
            return False

        # Recurse into children with tightened bounds:
        #
        # Left subtree: all values must be < root.val,
        # so the new ceil (upper bound) becomes root.val.
        # The floor stays the same (inherited from ancestors).
        #
        # Right subtree: all values must be > root.val,
        # so the new floor (lower bound) becomes root.val.
        # The ceil stays the same (inherited from ancestors).
        return (
            self.isValidBST(root.left, floor, root.val)
            and self.isValidBST(root.right, root.val, ceil)
        )

```

Note: I reordered the parameters to `(floor, ceil)` for readability — the reference solution uses `(ceil, floor)`, which is functionally identical but slightly confusing to read.

4. Complexity Analysis

Metric	Complexity	Explanation
Time	$O(n)$	Each node is visited exactly once.
Space	$O(h)$	Recursion stack depth equals tree height h . Best/balanced case: $O(\log n)$. Worst case (skewed tree): $O(n)$.

The reference says $O(n)$ space, which is the **worst-case** bound for a degenerate (linked-list-like) tree.

5. Alternative Approaches

A) In-order Traversal Check (very elegant)

An in-order traversal of a valid BST yields values in **strictly increasing order**. Just traverse and verify each value $>$ the previous.

```

class Solution:
    def isValidBST(self, root: Optional[TreeNode]) -> bool:
        self.prev = float("-inf") # last visited value during in-order walk

        def inorder(node):
            if not node:
                return True
            # 1. Visit left subtree
            if not inorder(node.left):
                return False
            # 2. Visit current node: must be strictly greater than previous
            if node.val <= self.prev:
                return False
            self.prev = node.val
            # 3. Visit right subtree
            if not inorder(node.right):
                return False
            return True

        return inorder(root)

```

```

        return False
    self.prev = node.val
    # 3. Visit right subtree
    return inorder(node.right)

return inorder(root)

```

• **Time:** $O(n)$, **Space:** $O(h)$.

B) Iterative In-order with a Stack

Same logic as (A) but avoids recursion using an explicit stack — useful to prevent stack overflow on deep trees.

```

class Solution:
    def isValidBST(self, root: Optional[TreeNode]) -> bool:
        stack, prev = [], float("-inf")
        while stack or root:
            # Go as far left as possible
            while root:
                stack.append(root)
                root = root.left
            root = stack.pop()
            # Process node in sorted order
            if root.val <= prev:
                return False
            prev = root.val
            root = root.right # move to right subtree
        return True

```

• **Time:** $O(n)$, **Space:** $O(h)$.

Summary of Patterns Used

1. **Bounds propagation (recursion with constraints)** — the core technique in the main solution.
2. **In-order traversal property of BSTs** — a powerful, reusable trick for many BST problems.
3. **Common pitfall** — only comparing a node to its immediate children is **wrong**; ancestral bounds matter.

Codex Solution

1. Intuition

A BST is not validated by checking only each node against its direct children.

For example:

```

    5
   / \
  1   6
   /
  3

```

The node `3` is less than `6`, so locally it looks fine as the left child of `6`.

But `3` is in the **right subtree of 5**, so it must be **greater than 5**. Therefore, this is not a valid BST.

Key insight

Every node has an allowed value range:

- For the root, the range is $(-\text{infinity}, \text{infinity})$.
- For a left child, the upper bound becomes the parent value.
- For a right child, the lower bound becomes the parent value.

So for every node:

```
lower_bound < node.val < upper_bound
```

The bounds must be **strict** because duplicates are not allowed in a valid BST.

2. Approach

We perform DFS while carrying the valid range for each node.

Pseudocode

```
function isValidBST(root):
    create stack containing (root, -infinity, infinity)

    while stack is not empty:
        node, lower, upper = stack.pop()

        if node is null:
            continue

        if node.val <= lower or node.val >= upper:
            return false

        push node.left with range (lower, node.val)
        push node.right with range (node.val, upper)

    return true
```

3. Python Solution

```
from typing import Optional

# Definition for a binary tree node.
# LeetCode already provides this class.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def isValidBST(self, root: Optional[TreeNode]) -> bool:
        """
        Validate whether a binary tree is a valid Binary Search Tree.

        Instead of only comparing a node with its direct children,
        we keep track of the allowed value range for every node.

        For each node:
            lower_bound < node.val < upper_bound

        The bounds are strict because duplicates are not allowed in a BST.
        """

        # Stack stores tuples of:
        # (current_node, minimum_allowed_value, maximum_allowed_value)
        # We use None to represent no bound instead of float("-inf") / float("inf").
        stack = [(root, None, None)]

        while stack:
            node, lower, upper = stack.pop()

            # Empty subtree is always valid.
            if node is None:
                continue

            # If there is a lower bound, node.val must be strictly greater.
            if lower is not None and node.val <= lower:
                return False

            # If there is an upper bound, node.val must be strictly smaller.
            if upper is not None and node.val >= upper:
                return False

            # Left subtree values must be:
            # greater than lower bound and strictly less than node.val.
            stack.append((node.left, lower, node.val))
```

```
# Right subtree values must be:
# strictly greater than node.val and less than upper bound.
stack.append((node.right, node.val, upper))

# If every node satisfies its valid range, this is a BST.
return True
```

4. Complexity Analysis

Let n be the number of nodes in the tree.

Time Complexity

$O(n)$

Each node is visited exactly once.

Space Complexity

$O(h)$

Where h is the height of the tree.

- For a balanced tree, $h = \log n$, so space is $O(\log n)$.
- For a completely skewed tree, $h = n$, so space is $O(n)$.

Because this solution uses an explicit stack, it avoids Python recursion depth issues for very deep trees.

5. Alternative Approaches

1. Recursive DFS with bounds

Same idea, but implemented recursively.

```
class Solution:
    def isValidBST(self, root: Optional[TreeNode]) -> bool:
        def dfs(node, lower, upper):
            if node is None:
                return True

            if lower is not None and node.val <= lower:
                return False

            if upper is not None and node.val >= upper:
                return False

            return dfs(node.left, lower, node.val) and dfs(node.right, node.val, upper)

        return dfs(root, None, None)
```

This is clean and common, but may hit recursion depth limits on very skewed trees in Python.

2. Inorder traversal

A valid BST has an inorder traversal that is strictly increasing.

left subtree -> node -> right subtree

During traversal, keep track of the previous visited value. If the current value is not greater than the previous value, return `False`.

This also runs in:

Time: $O(n)$
Space: $O(h)$

The bounds-based DFS is often preferred because it directly follows the BST definition.